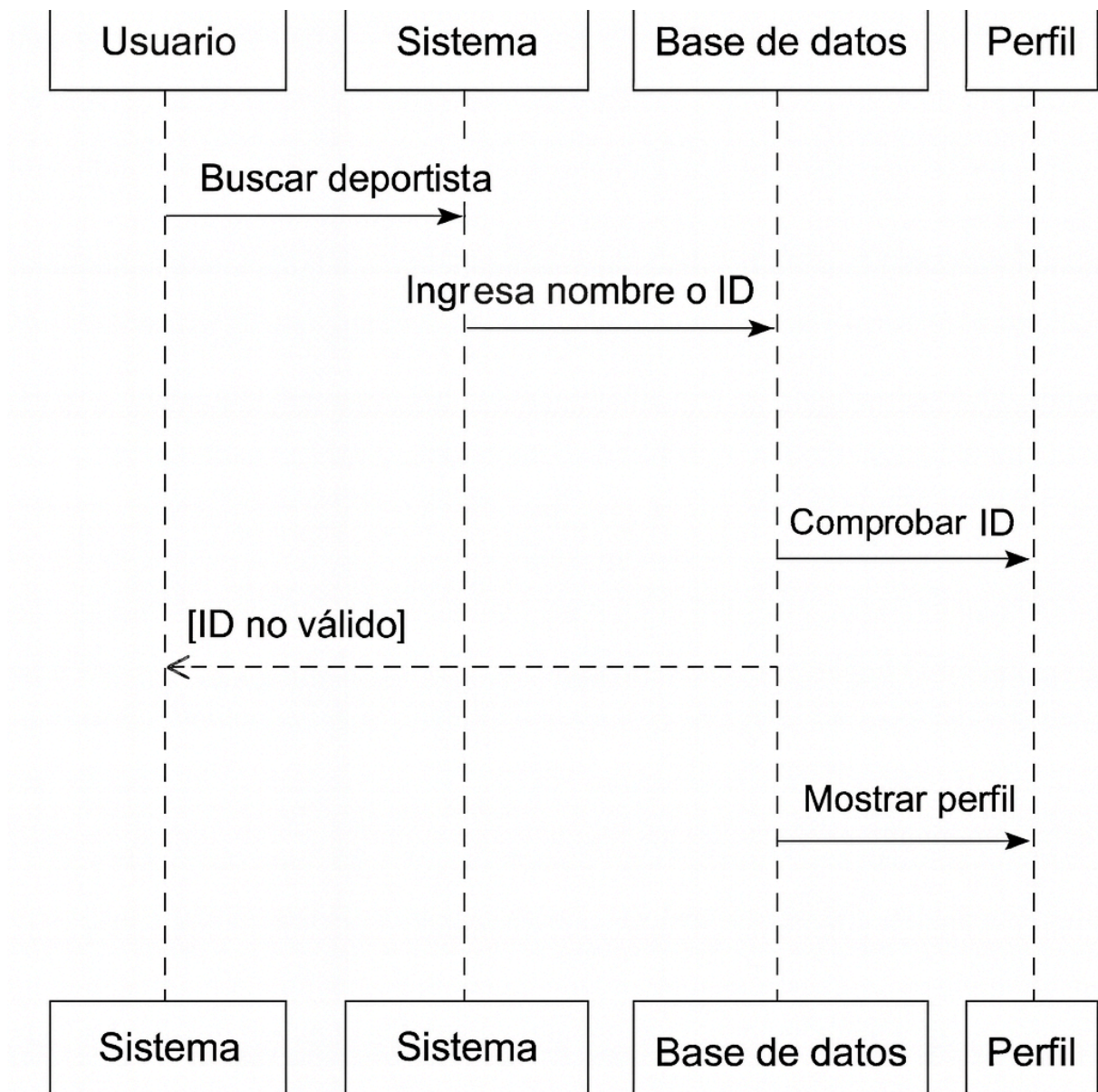
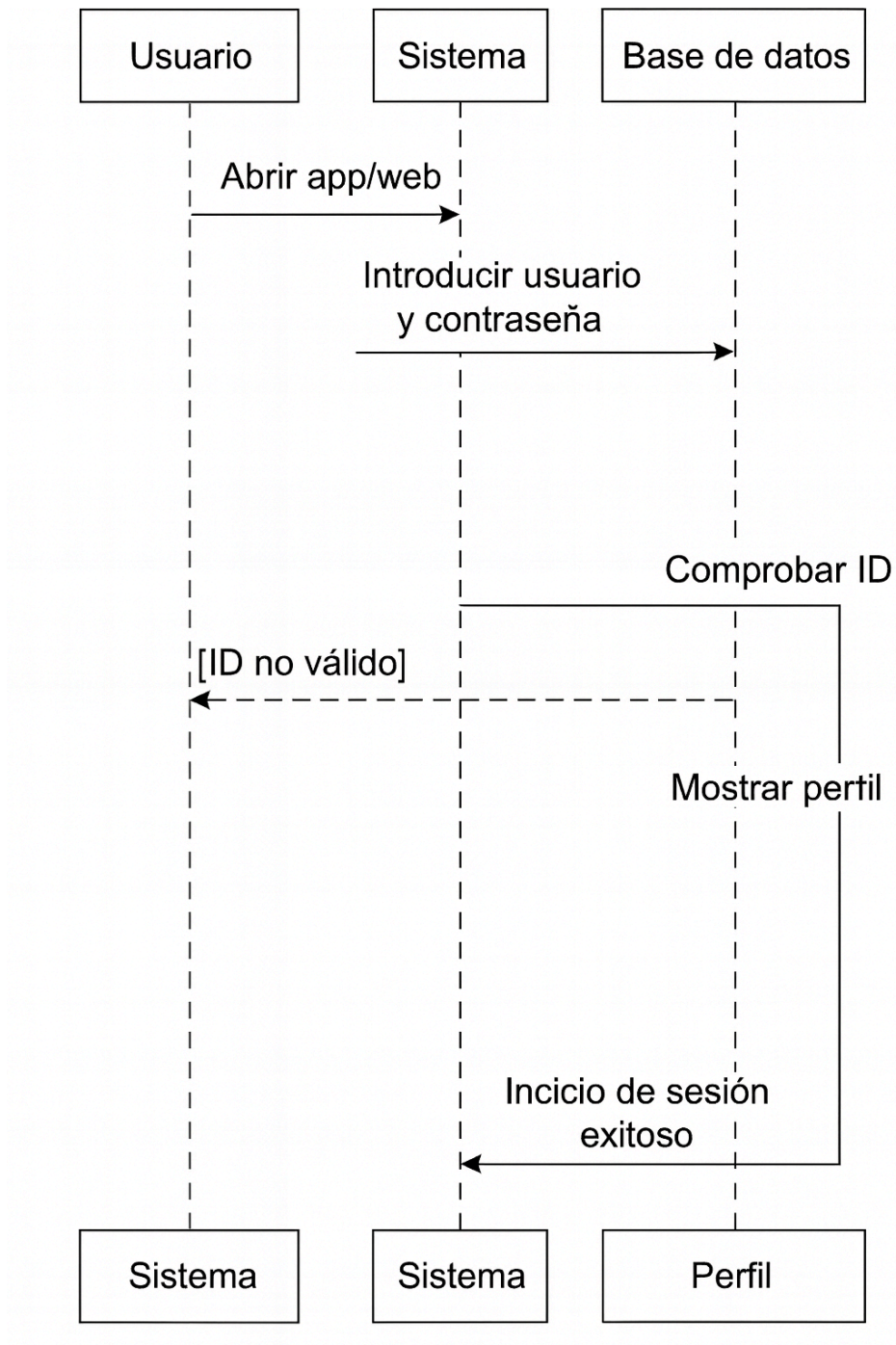


1. Diagramas de secuencia.

- Diagrama de secuencia “Encontrar a un corredor”.



- Diagrama de secuencia “Inicio sesión”



2. Investigación y selección tecnológica

Front-end

- **Framework principal: React + TypeScript + [Next.js](#)**
 - **Por qué:** Trailconnect seguramente necesitará SEO fuerte (para posicionar rutas y contenidos), renderizado rápido y soporte para mapas interactivos. Next.js ofrece SSR/SSG y optimización automática.
- **Bibliotecas clave:**
 - **Leaflet / Mapbox GL JS:** para mapas interactivos y visualización de rutas.
 - **Tailwind CSS:** para diseño rápido y consistente.
 - **React Query / TanStack Query:** para gestionar datos remotos (APIs de rutas, usuarios).
 - **Ventajas:** ecosistema maduro, gran comunidad, integración con librerías de mapas y geolocalización.
 - **Desventajas:** curva de aprendizaje si el equipo no está familiarizado con React/Next.

Back-end

- **Lenguaje y framework: Node.js + NestJS**
 - **Por qué:** APIs REST/GraphQL para usuarios, rutas, reseñas y tiempo real (chat, notificaciones). NestJS aporta modularidad y escalabilidad.
- **Arquitectura:**
 - **Microservicios** ligeros (usuarios, rutas, comunidad, notificaciones).

- **WebSockets** para tiempo real (seguimiento en vivo, alertas).
- **Alternativas:** Django (Python) si se prioriza integración con análisis de datos/IA; Spring Boot si se busca robustez enterprise.

Base de datos

- **Principal: PostgreSQL**
 - Por qué: Trailconnect manejará datos estructurados (usuarios, rutas, reseñas, eventos). PostgreSQL ofrece integridad ACID y soporte para datos geoespaciales con PostGIS.
- **Complementaria: MongoDB**
 - **Por qué:** para almacenar datos flexibles como registros de actividad, logs, o contenido generado por usuarios (comentarios, fotos).
 - **Ventajas:**
 - PostgreSQL + PostGIS: consultas geoespaciales potentes (distancias, áreas, rutas).
 - MongoDB: flexibilidad y escalado horizontal.
 - **Desventajas:**
 - Operar dos motores implica mayor complejidad de mantenimiento.
 - PostgreSQL requiere diseño cuidadoso de índices para rendimiento en consultas geográficas.

Justificación final para Trailconnect

- **Front-end:** React + Next.js + Tailwind + Mapbox → SEO, rendimiento y mapas interactivos.
- **Back-end:** Node.js + NestJS → APIs escalables y soporte en tiempo real.

- **Base de datos:** PostgreSQL (con PostGIS) para datos geográficos y relacionales; MongoDB para datos flexibles.

Este stack equilibra **rendimiento, escalabilidad y experiencia de usuario**, clave para una plataforma de rutas y comunidad como Trailconnect.